

FAST: Enhancing Federated Learning Through Adaptive Data Sampling and Local Training

Zhiyuan Wang^{ID}, Hongli Xu^{ID}, *Member, IEEE*, Yang Xu^{ID}, *Member, IEEE*, Zhida Jiang^{ID},
Jianchun Liu^{ID}, *Member, IEEE*, and Suo Chen^{ID}

Abstract—The emerging paradigm of federated learning (FL) strives to enable devices to cooperatively train models without exposing their raw data. In most cases, the data across devices are non-independently and identically distributed in FL. Thus, the local models trained over different data distributions will inevitably deviate from the global optima, which induces optimization inconsistency and even hurts global convergence. Moreover, the resource-constrained devices with heterogeneous training capacities (e.g., computing and communication) further slow down the convergence rate. To this end, we introduce an FL framework with adaptive data sampling and local training, namely FAST. Specifically, even without devices' private data distributions, FAST enables each device to sample different rates of data points from each of its local classes to rebuild a dataset for training, thus adjusting the convergence direction of the aggregated global model to be closer to the global optima. The theoretical analysis shows that the convergence bound depends on the sampling rates as well as the number of local iterations executed on the sampled data. To achieve resource-effective and convergence-guaranteed FL, we then design an online learning algorithm that jointly optimizes the data sampling and local training strategies so as to encourage the decrease of global loss under the given time budget. Extensive experiments on physical and simulated environments show that, FAST improves the model accuracy by about 1.55%-6.78% given the same time budget, and accelerates training by about 1.39-5.89× with the same target accuracy, compared with the baselines.

Index Terms—Data sampling, device heterogeneity, edge computing, federated learning (FL), non-IID.

I. INTRODUCTION

RECENTLY, billions of end devices (e.g., smartphones, vehicles) around the world produce a tremendous amount of data per second [1], [2]. Coupled with the rise of deep neural networks (DNNs), the wealth of those collected data opens up countless possibilities for meaningful and advanced AI applications [3], [4]. Traditionally, the data generated at

network edge will be uploaded and processed centrally in the cloud. However, considering the massive communication cost as well as the privacy concern associated with raw data movement, such centralized training operation becomes more and more infeasible [5], [6].

To tackle such issues, federated learning (FL), which enables devices to collaboratively build a shared model while preserving private data locally [7], [8], comes into being in edge computing. In general, FL comprises two components, i.e., the parameter server and devices [9], [10]. Devices cooperatively train models over their local data in a synchronous manner. After each device performs local training, the updated local model will be forwarded to the parameter server for global aggregation. Under this framework, the parameter server maintains the globally shared up-to-date model to solve large-scale learning problems, which can protect privacy [8], and relieve network burden [5], [7], compared with traditional centralized training.

However, to realize the benefits of FL, we still face some challenges due to the intrinsic features of edge computing.

1) *Non-IID data among devices*: In FL, since the training data on the individual devices always depend on their local environment and usage pattern, the local data distributions of different devices will vary heavily, which inevitably incurs the non-independently and identically distributed (Non-IID) issue [11], [12] and deteriorates the training efficiency of FL. Specifically, each device's local data distribution is often not a good representative of the global distribution [13], [14], and the local models of different devices may easily converge in different directions, resulting in global accuracy degradation [13], [15].

2) *Limited and heterogeneous resources on devices*: The resources, including communication and computing, are always limited and heterogeneous among devices. For example, the communication between devices and the parameter server may travel via various types of networks, e.g., cellular networks, and the Internet, leading to different communication capacities [1], [16]. For computing, since model training imposes heavy computing requirements and the amount of data among devices is imbalanced [17], the performance gap between heterogeneous devices is further widened, which will severally slow down the convergence rate [5], [9].

As these issues arise from the widely distributed heterogeneous devices with their private local data, many studies adopt different device or data management approaches to improve the training performance of FL, which can be roughly divided into the following three categories:

Manuscript received 1 March 2023; revised 30 September 2023; accepted 15 November 2023. Date of publication 20 November 2023; date of current version 18 December 2023. This work was supported in part by the National Science Foundation of China (NSFC) under Grants 62102391, 62132015 and 61936019, and in part by the Jiangsu Province Science Foundation for Youths under Grant BK20210122. Recommended for acceptance by T. Estrada. (Corresponding author: Yang Xu.)

The authors are with the University of Science and Technology of China, Hefei, Anhui 230027, China, and also with Suzhou Institute for Advanced Study, University of Science and Technology of China, Suzhou, Jiangsu 215123, China (e-mail: cswangzy@mail.ustc.edu.cn; xuhongli@ustc.edu.cn; xuyangcs@ustc.edu.cn; zdjiang@mail.ustc.edu.cn; jcliu17@ustc.edu.cn; chen-suo@mail.ustc.edu.cn).

Digital Object Identifier 10.1109/TPDS.2023.3334398

1) *Cross-device data sharing*: To alleviate the effect of Non-IID data on model convergence, one straightforward way is to perform data migration among devices or distribute globally shared data to devices from some clouds [13], [18], [19]. In this way, the data distribution difference among devices will be reduced, and the training speed among heterogeneous devices can be balanced by moving some data from weak devices to some strong devices, at the expense of privacy concerns and increased communication overhead. Due to the increasingly stringent privacy laws (e.g., European Commission's General Data Protection Regulation [20]), many raw data in devices or clouds are not allowed to be exposed [8]. Besides, as the devices' communication resource is limited in edge computing [21], cross-device data sharing becomes more unsuitable [22].

2) *Device-level data sampling*: This category chooses only a fraction of devices participating in training, also called *client selection* [23]. Some approaches [23], [24], [25], [26] choose faster devices with sufficient resources for handling device heterogeneity, while ignoring their different data distributions, i.e., deteriorating training efficiency in the presence of Non-IID data. Some other works [14], [27] utilize weight divergence to recognize the Non-IID degrees of devices, and choose those with lower degrees to counterbalance the bias induced by Non-IID data. However, considering the limited and heterogeneous resource of devices, the training speed of those works may be seriously restricted, i.e., each training epoch only progresses as fast as the slowest devices (i.e., synchronization cost [10]). Besides, it is estimated that the number of epochs for convergence is nearly doubled when the number of devices is halved [28], [29], resulting in considerable training time lags.

3) *Sample-level data sampling*: With this approach, each device selects a fraction of data points from its entire local dataset for training. To alleviate the device heterogeneity, Feng et al. [30] enable each device to train a randomly selected subset of data points for training. On the basis of this work, the researchers [31], [32], [33], [34] mainly target to select some relevant data samples with smaller loss or higher gradient variation on the model and discard the irrelevant ones, and clarify that this operation enables potential in handling Non-IID issue. However, the data sampling criterion may seriously influence the training performance [32], e.g., how many data samples should be discarded or maintained for training is always uncertain in those works. Besides, the optimization directions of different devices may also be extremely different since their different data distributions have not been taken into consideration, which inevitably hurts global convergence [11].

To effectively deal with the Non-IID issue as well as the resource limitation in edge computing, in this paper, we design a novel FL framework, called FAST. Specifically, to reduce the data distribution difference among devices and alleviate the synchronization cost of the whole system, FAST enables each device to sample different rates of data points from each of its local classes for training. In this way, the local models can be optimized in a direction closer to the global optima, and the training speed will be effectively accelerated. To further improve the generality of our framework, FAST also integrates adaptive local training with data sampling. If devices only have few data

samples in some special classes (i.e., the local distribution may not be closer to the global distribution enough after sampling), FAST adaptively reduces the number of local iterations during per-epoch local training, to avoid the local model to deviate from the global optima before global aggregation. Otherwise, more local iterations will be executed for reducing the global aggregation frequency so as to save the scarce communication resource.

In essence, from the convergence performance perspective, data sampling influences the optimization direction while the number of local iterations influences the step size. In terms of the resource consumption, a larger size of the sampled data consumes more computing resource while a larger number of local iterations saves more communication resource. By jointly optimizing the data sampling and local training, FAST will achieve resource-effective and convergence-guaranteed model training. The main contributions are summarized as follows:

- 1) We propose a novel FL framework, called FAST, to address the Non-IID issue as well as the constrained and heterogeneous resource of devices in edge computing. Furthermore, we theoretically prove that FAST can provide the convergence guarantee for model training.
- 2) To enhance the training performance, we design an online learning algorithm that jointly optimizes the data sampling and local training strategies so as to encourage the decrease of global loss under the given time budget without accessing devices' private data distributions.
- 3) We conduct extensive experiments on physical and simulated environments. The experimental results show that FAST improves the accuracy by 1.55%-6.78% given the same time budget, and accelerates training by 1.39-5.89 $\times$ with the same target accuracy, compared with baselines.

The rest of this paper is organized as follows. We propose our framework in Section II. In Section III, we theoretically prove its convergence performance. Section IV provides the algorithm design. Section V evaluates the performance of our proposed algorithms. We review the related work in Section VI, and conclude this paper in Section VII.

II. PRELIMINARIES

A. System Model

We consider a typical FL system [7], [8] with one parameter server and a set of devices $\mathcal{U} = \{u_1, \dots, u_m\}$, where $m = |\mathcal{U}|$ is the total number of devices. In addition, the local dataset of each device $u \in \mathcal{U}$ is denoted as $D_u = \{(\mathbf{x}_i, y_i)\}_{i=1}^{|D_u|}$ where $\mathbf{x}_i$ is the i -th data sample, y_i is the corresponding label and $|D_u|$ is the local data size. To cooperatively solve a learning problem, each device trains a local model over its local dataset and periodically synchronizes its model with the parameter server to reach the global model. Specifically, we consider a C class classification problem in FL, i.e., $y_i \in \{1, 2, \dots, C\}, \forall i$. For device u , the total number of data samples with label $c \in \{1, 2, \dots, C\}$ is then calculated as $N_u^c = \sum_{i=1}^{|D_u|} \mathbb{I}_{y_i=c}$, where $\mathbb{I}$ is the indicator function. The local data distribution of each device u is then expressed as $P_u = \{p_u^1, p_u^2, \dots, p_u^C\}$, where $p_u^c = N_u^c / |D_u|$ and

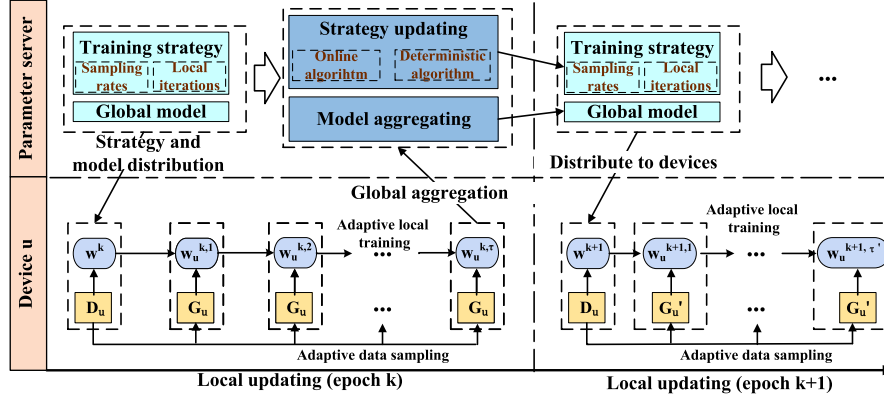


Fig. 1. Training workflow of FAST between the parameter server and an arbitrary device u .

$\sum_{c=1}^C p_u^c = 1$. Similarly, let $P = \{p^1, p^2, \dots, p^C\}$ be the global data distribution among the local dataset of all distributed devices, i.e., $p^c = \sum_{u \in \mathcal{U}} N_u^c / \sum_{u \in \mathcal{U}} |D_u|$. Note that the devices' data distribution will not be exposed to the parameter server for privacy protection in FAST, and will be learned or estimated by the proposed online learning algorithm (i.e., MAB) instead. For the synchronous training in FAST, let K be the total number of global aggregations, and the number of local iterations among two consecutive global aggregations is denoted as τ . Each device will train the model over its local dataset in each local iteration. Through iterative local training and global aggregation, the devices can eventually obtain a well-trained global model and benefit from the intelligent learning service.

B. System Workflow of FAST

We then illustrate the workflow of FAST, including strategy and model distribution, local training and global aggregation, and Fig. 1 presents the overall design.

Strategy and model distribution: To handle the Non-IID issue as well as the resource constraint in edge computing, FAST enables each device u to sample a subset G_u from its entire local dataset (i.e., $G_u \subseteq D_u$) in each local iteration according to the assigned data sampling strategy, and execute this procedure for τ local iterations totally in each training epoch. Specifically, the parameter server adaptively determines the sampling rate $\bar{q}_u^c$ of class c for each device u to form a new training dataset with the distribution $Q_u = \{q_u^1, q_u^2, \dots, q_u^C\}$, where the size of the sampled data and new data distribution are computed by

$$|G_u| = \sum_{c=1}^C N_u^c \cdot \bar{q}_u^c, \quad q_u^c = \frac{N_u^c \cdot \bar{q}_u^c}{|G_u|}. \quad (1)$$

After distributing the corresponding training strategy (i.e., $\bar{q}_u^c$ and τ) and the global model to each device by the parameter server, the local updating procedure will be triggered. Note that such training strategy will be updated adaptively in each epoch based on devices' current training performance.

Local updating: In conventional FL [9], [13], each device always associates with a local loss function f_u based on the inherent data distribution P_u of the local dataset D_u . Specifically, the population loss f_u with the widely used cross-entropy

function [35] is defined as

$$\begin{aligned} f_u(w_u^k) &= \mathbb{E}_{\mathbf{x}_i, y_i \sim P_u} \left[- \sum_{c=1}^C \mathbb{I}_{y_i=c} \log f_c(\mathbf{x}_i, w_u^k) \right] \\ &= \sum_{c=1}^C p_u^c \mathbb{E}_{\mathbf{x}_i | y_i=c} \left[\log \frac{1}{f_c(\mathbf{x}_i, w_u^k)} \right], \end{aligned} \quad (2)$$

where w_u^k is the local model of device u at epoch $k \in \{1, 2, \dots, K\}$ and $f_c(\mathbf{x}_i, w_u^k)$ is the predicted probability of the c -th class outputted by the model w_u^k for the input data point $\mathbf{x}_i$. However, through data sampling in FAST, the corresponding loss of device u defined on G_u is

$$f_u(w_u^k) = \sum_{c=1}^C q_u^c \mathbb{E}_{\mathbf{x}_i | y_i=c} \left[\log \frac{1}{f_c(\mathbf{x}_i, w_u^k)} \right]. \quad (3)$$

In each epoch k , the learning objective on each device u is to reach a local model w_u^k that minimizes the loss $f_u(w_u^k)$, i.e., $\min_{w_u^k} f_u(w_u^k)$. To achieve this target, device u iteratively updates the local model using gradient descent [36] and this process will repeat for τ times at each epoch k , i.e.,

$$w_u^{k, \tau'+1} = w_u^{k, \tau'} - \eta \nabla f_u(w_u^{k, \tau'}), \quad 0 \leq \tau' < \tau, \quad (4)$$

where $w_u^{k, \tau'}$ is the local model of device u at round k after τ' local iterations ($w_u^{k, 0} = w_u^k$), η is the learning rate and $\nabla f_u(w_u^{k, \tau'})$ is the gradient of the loss function (defined on the dataset G_u) with respect to $w_u^{k, \tau'}$.

Global aggregation: After local training, each device will forward the updated local model to the parameter server for global aggregation, i.e.,

$$w^{k+1} = \frac{1}{m} \sum_{u \in \mathcal{U}} w_u^{k, \tau} \quad (5)$$

where w^{k+1} is the global model at epoch $k+1$. This up-to-date global model will be broadcast to all devices for further local training. The above procedure will be repeated until the global loss defined on the global data distribution, i.e., $f(w^K) = - \sum_{c=1}^C p^c \mathbb{E}_{\mathbf{x}_i | y_i=c} [\log f_c(\mathbf{x}_i, w_u^K)]$, is minimized after K epochs, or the resource budgets are exceeded. Since the overall datasets $\cup_{u \in \mathcal{U}} D_u$ among all devices are unavailable, we

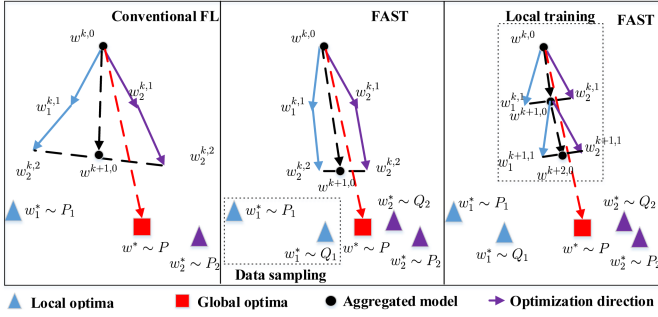


Fig. 2. Illustration of the model optimization procedure in the parameter space with Non-IID data.

generally minimize the empirical loss function instead in FL, i.e., $f(w^K) = \frac{1}{m} \sum_{u \in \mathcal{U}} f_u(w^K)$.

For clear illustration, we give an example in Fig. 2. Given one parameter server and 2 devices, each device has different data distributions. In conventional FL [7], [13], each device trains the local model based on its entire local dataset, and the local model may simply get stuck at local optima (which can be far from the global optima) due to the inherent data distribution. As a result, the global model deviates from optimal value w^* especially when the number of local iterations is large during local training [11], [37], resulting in performance deterioration, as shown in the left plot of Fig. 2 with $\tau = 2$. However, by adopting data sampling, the actual data distribution of each device can be modified, and thus the optimization direction of local model can be adjusted, resulting in a better training performance of the global model in the middle plot of Fig. 2. Moreover, considering the case in the right plot of Fig. 2, in which some devices may only have few or even no data samples in some special classes. At this moment, we intuitively reduce the number of local iterations, and thus the local model will not deviate from the global optima during local training. To summarize, FAST jointly optimizes the data sampling and local training for achieving resource-effective and convergence-guaranteed FL in edge computing. As such two factors collaborate to influence the training performance of FAST, we will provide the problem definition and the convergence analysis in the following subsections, correspondingly.

C. Problem Definition

In FL systems, devices are usually equipped with limited and heterogeneous resources, leading to different training speeds. Without loss of generality, we take two major kinds of resources, i.e., computing and communication, into consideration. The computing delay of each device is associated with its sampled data size, the number of local iterations and its training capacities (e.g., CPU frequency), while the communication delay mainly depends on the model size, the link speed as well as the frequency of global aggregation. Specifically, for each device u at epoch k , let $t_{u,cp}^k$ be the time consumption for processing one data sample and $t_{u,cm}^k$ be the per-epoch communication delay for model transmission. Note that each type of time consumption can be extremely different among devices due to device heterogeneity and network dynamics. Furthermore, since the operations of

computing and communication at each device can be performed in parallel, the per-epoch time consumption t^k depends on the slowest participating device (i.e., synchronization cost), which is calculated by $t^k = \max_{u \in \mathcal{U}} \{\tau |G_u| t_{u,cp}^k + t_{u,cm}^k\}$. To accelerate the training procedure and guarantee the convergence performance, we formulate the problem as follows,

$$\begin{aligned} & \min_{K \in \{1, 2, \dots\}} f(w^K) \\ \text{s.t. } & \begin{cases} \sum_{k=1}^K t^k \leq T \\ t^k = \max_{u \in \mathcal{U}} \{\tau |G_u| t_{u,cp}^k + t_{u,cm}^k\}, \quad \forall k \\ |G_u| q_u^c \leq N_u^c, \quad \forall u, c \\ \bar{q}_u^c \in [0, 1], \tau \in \{1, 2, \dots\} \quad \forall u, c \end{cases} \end{aligned} \quad (6)$$

The first set of inequalities indicates that the total time consumption after K epochs should not exceed the time budget, i.e., T . The second set of equalities expresses the synchronous training procedure in FAST. The third set of constraints denotes the size of sampled data in each class is smaller than that of the original dataset in this class. The objective is to minimize the global loss of the learning task.

In fact, it is impossible to directly solve (6) because there does not exist a closed-form expression among $\bar{q}_u^c, \forall u, c, \tau$ and $f(w^K)$, and these parameters are always interdependent with complex relationship. For example, data sampling $\bar{q}_u^c$ influences the optimization direction of local model while the number of local iterations τ affects the step size, which jointly determine the training performance, i.e., $f(w^K)$. Hence, we analyze the convergence bound of FAST, then use this bound to approximately solve (6) and propose a control algorithm for adaptively choosing the optimal values of $\bar{q}_u^c, \forall u, c$, and τ .

III. CONVERGENCE ANALYSIS

We analyze the convergence performance of FAST by comparing it with the centralized training, in which all datasets among devices $\cup_{u \in \mathcal{U}} D_u$ are aggregated together with a distribution $P = \{p^1, p^2, \dots, p^C\}$. Therefore, the loss function of centralized training at each epoch k can be denoted as

$$f(\bar{w}^k) = \sum_{c=1}^C p^c \mathbb{E}_{\mathbf{x}_i | y_i = c} \left[\log \frac{1}{f_c(\mathbf{x}_i, \bar{w}^k)} \right]. \quad (7)$$

Accordingly, the learning problem becomes $\min_{\bar{w}^k} f(\bar{w}^k)$. In the field of machine learning, we cannot deny that centralized training always outperforms distributed machine learning (e.g., federated learning) [5], [9], [38]. We analyze the model divergence between FAST and centralized training after K training epochs, i.e., $\|w^K - \bar{w}^K\|$, where $\|\cdot\|$ is the vector L_2 normal, by considering the sampled data distribution Q_u of each device u and the number of local iterations τ . Specifically, the less the model divergence between FAST and centralized training, the better the FAST's training performance. Given the widely used assumption [9], [10], i.e., the loss f_u is with $\mathcal{L}$ -Lipschitzian gradients,

$$\|\nabla f_u(w_1) - \nabla f_u(w_2)\| \leq \mathcal{L} \|w_1 - w_2\|, \forall w_1, w_2, u, \quad (8)$$

we theoretically analyze the convergence performance in the following three steps.

First, considering the global aggregation step in (5) of FAST, we derive that

$$\begin{aligned}
\|w^k - \bar{w}^k\| &= \left\| \frac{1}{m} \sum_{u \in \mathcal{U}} w_u^{k,\tau} - \bar{w}^{k,\tau} \right\| \\
&= \left\| \frac{1}{m} \sum_{u \in \mathcal{U}} \left(w_u^{k,\tau-1} + \eta \sum_{c=1}^C q_u^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, w_u^{k,\tau-1})] \right) \right. \\
&\quad \left. - \left(\bar{w}^{k,\tau-1} + \eta \sum_{c=1}^C p^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-1})] \right) \right\| \\
&\leq \frac{1}{m} \sum_{u \in \mathcal{U}} \|w_u^{k,\tau-1} - \bar{w}^{k,\tau-1}\| \\
&\quad + \eta \underbrace{\left\| \frac{1}{m} \sum_{u \in \mathcal{U}} \sum_{c=1}^C q_u^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, w_u^{k,\tau-1})] \right\|}_{A_1} \\
&\quad - \underbrace{\sum_{c=1}^C p^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-1})]}_{A_2} \quad (9)
\end{aligned}$$

For $\|A_1 - A_2\|$ in (9), we can further bound it as

$$\begin{aligned}
\|A_1 - A_2\| &= \left\| \frac{1}{m} \sum_{u \in \mathcal{U}} \sum_{c=1}^C q_u^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, w_u^{k,\tau-1})] \right. \\
&\quad \left. - \frac{1}{m} \sum_{u \in \mathcal{U}} \sum_{c=1}^C q_u^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-1})] \right. \\
&\quad \left. + \frac{1}{m} \sum_{u \in \mathcal{U}} \sum_{c=1}^C q_u^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-1})] \right. \\
&\quad \left. - \sum_{c=1}^C p^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-1})] \right\| \\
&\leq \frac{1}{m} \sum_{u \in \mathcal{U}} \sum_{c=1}^C q_u^c \|\nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, w_u^{k,\tau-1})] \\
&\quad - \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-1})]\| \\
&\quad + \frac{1}{m} \sum_{u \in \mathcal{U}} \left\| \sum_{c=1}^C (q_u^c - p^c) \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-1})] \right\| \\
&\leq \frac{\mathcal{L}}{m} \sum_{u \in \mathcal{U}} \|w_u^{k,\tau-1} - \bar{w}^{k,\tau-1}\| + \frac{\varepsilon}{m} \sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\|. \quad (10)
\end{aligned}$$

Note that in the last step of (10), we apply the $\mathcal{L}$ -Lipschitzian assumption to the first term, and let $\varepsilon > 0$ be the upper bound of $\|\nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-1})]\|$, i.e., $\varepsilon = \max_{c=1}^C \|\nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-1})]\|$. Hence, plugging this upper bound into (9), we have

$$\|w^k - \bar{w}^k\| \leq \frac{1 + \eta\mathcal{L}}{m} \sum_{u \in \mathcal{U}} \|w_u^{k,\tau-1} - \bar{w}^{k,\tau-1}\|$$

$$+ \frac{\eta\varepsilon}{m} \sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\|. \quad (11)$$

Second, for the local training on an arbitrary device u at epoch k in (4), we derive the bound as follows,

$$\begin{aligned}
&\|w_u^{k,\tau-1} - \bar{w}^{k,\tau-1}\| \\
&= \left\| \left(w_u^{k,\tau-2} + \eta \sum_{c=1}^C q_u^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, w_u^{k,\tau-2})] \right) \right. \\
&\quad \left. - \left(\bar{w}^{k,\tau-2} + \eta \sum_{c=1}^C p^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-2})] \right) \right\| \\
&\leq \|w_u^{k,\tau-2} - \bar{w}^{k,\tau-2}\| + \eta \left\| \sum_{c=1}^C q_u^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} \right. \\
&\quad \times [\log f_c(\mathbf{x}_i, w_u^{k,\tau-2})] \\
&\quad \left. - \sum_{c=1}^C p^c \nabla \mathbb{E}_{\mathbf{x}_i|y_i=c} [\log f_c(\mathbf{x}_i, \bar{w}^{k,\tau-2})] \right\| \\
&\leq (1 + \eta\mathcal{L}) \|w_u^{k,\tau-2} - \bar{w}^{k,\tau-2}\| + \eta\varepsilon \sum_{c=1}^C \|q_u^c - p^c\| \\
&\dots \\
&\leq (1 + \eta\mathcal{L})^{\tau-1} \|w^{k-1} - \bar{w}^{k-1}\| \\
&\quad + \sum_{i=0}^{\tau-2} (1 + \eta\mathcal{L})^i \eta\varepsilon \sum_{c=1}^C \|q_u^c - p^c\| \\
&= (1 + \eta\mathcal{L})^{\tau-1} \|w^{k-1} - \bar{w}^{k-1}\| \\
&\quad + \frac{(1 + \eta\mathcal{L})^{\tau-1} - 1}{\mathcal{L}} \varepsilon \sum_{c=1}^C \|q_u^c - p^c\|. \quad (12)
\end{aligned}$$

The derivation process of the third step in (12) is similar to that of (10), and has been omitted. Since each training epoch k consists of τ local iterations on each device u (i.e., $w^{k-1} = w_u^{k,0}$), we obtain the results in the last two steps.

Third, putting the upper bound in (12) back into (11), we can derive the relationship between the convergence bounds of two consecutive training epochs.

$$\begin{aligned}
\|w^k - \bar{w}^k\| &\leq (1 + \eta\mathcal{L})^\tau \|w^{k-1} - \bar{w}^{k-1}\| \\
&\quad + \frac{[(1 + \eta\mathcal{L})^\tau - 1]\varepsilon}{m\mathcal{L}} \sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\|. \quad (13)
\end{aligned}$$

For simplicity, let $A_3 = \frac{[(1 + \eta\mathcal{L})^\tau - 1]\varepsilon}{m\mathcal{L}} \sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\|$. Given the same initialized model for FAST and centralized training (i.e., $w^0 = \bar{w}^0$), we finally obtain the convergence bound after K training epochs.

$$\begin{aligned}
&\|w^K - \bar{w}^K\| \\
&\leq (1 + \eta\mathcal{L})^\tau \|w^{K-1} - \bar{w}^{K-1}\| + A_3 \\
&\dots
\end{aligned}$$

$$\begin{aligned}
&\leq (1 + \eta\mathcal{L})^{\tau K} \|w^0 - \bar{w}^0\| + \sum_{i=0}^K (1 + \eta\mathcal{L})^{\tau i} A_3 \\
&= \frac{(1 + \eta\mathcal{L})^{\tau K + \tau} - 1}{(1 + \eta\mathcal{L})^{\tau} - 1} \frac{[(1 + \eta\mathcal{L})^{\tau} - 1]\varepsilon}{m\mathcal{L}} \sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\|.
\end{aligned} \tag{14}$$

Theorem 1: After K training epochs, the model divergence between FAST and centralized training is bounded by

$$\begin{aligned}
&\|w^K - \bar{w}^K\| \\
&= \frac{(1 + \eta\mathcal{L})^{\tau K + \tau} - 1}{(1 + \eta\mathcal{L})^{\tau} - 1} \frac{[(1 + \eta\mathcal{L})^{\tau} - 1]\varepsilon}{m\mathcal{L}} \sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\|.
\end{aligned} \tag{15}$$

Note that this bound is related to the distance between the sampled data distribution on each device and the global distribution (i.e., $\sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\|$), which can be minimized by adopting the optimal data sampling result. Furthermore, we observe that the bound depends on the number of local iterations τ . A smaller τ contributes to the reduction of the model divergence between FAST and centralized training, while it consumes tremendous communication resource due to the frequent global aggregations. Finally, we note that the model divergence increases with the total number of training epochs K when $\sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\| \neq 0$. Such model divergence comes from two parts, i.e., the model divergence after $K - 1$ epochs and the model divergence induced by the data distribution difference. As the amplification of the model divergence, the model in FL will eventually converge to lower accuracy than that of centralized training. Our target is to reduce or even erase this negative impact by adaptive data sampling and local training. Due to its hardness, we will propose an online learning algorithm, and the details of our solution will be described in the next section.

IV. ALGORITHM DESIGN

To develop data sampling and local training strategy so as to accelerate FL, in this section, we formally propose the algorithm for solving the problem in (6) in three steps. 1) We transform the problem to provide a closed-form expression among all parameters to be solved. 2) We propose the MAB based online algorithm to adaptively provide the data sampling result in each epoch without any prior knowledge of the devices' private local data distributions. 3) Given the sampling rates, the number of iterations τ during local training that minimizes the objective can then be calculated.

A. Problem Transformation

Given a loss function f , its optimal value $f(w^*)$ is always a constant. Therefore, we can rewrite the objective function $f(w^K)$ in (6) as $f(w^K) - f(w^*)$. Next, we will replace $f(w^K) - f(w^*)$ by the approximate function which is related to $|G_u|$, Q_u , τ and K in three steps, where $|G_u|$ and Q_u are directly determined by the sampling rates of each device u .

First, based on the convergence rate in many previous works [9], [19], [29], we obtain that,

$$f(w^K) - f(w^*) \propto \frac{1}{K}, \tag{16}$$

which denotes the model will converge with a rate of $\mathcal{O}(\frac{1}{K})$.

Second, by $\mathcal{L}$ -Lipschitzian assumption, we have

$$\begin{aligned}
f(w^K) - f(w^*) &\leq \nabla f(w^*)^\top (w^K - w^*) + \frac{\mathcal{L}}{2} \|w^K - w^*\|^2 \\
&= \frac{\mathcal{L}}{2} \|w^K - w^*\|^2,
\end{aligned} \tag{17}$$

where $\nabla f(w^*) = 0$. As the model divergence $\|w^K - w^*\|^2$ between the global model and the optimal model increases, the convergence performance gets worse. Using the triangle inequality, we have $\|w^K - w^*\| \leq \|w^K - \bar{w}^K\| + \|\bar{w}^K - w^*\|$. The upper bound of $\|w^K - \bar{w}^K\|$ is shown in Theorem 1. For $\|\bar{w}^K - w^*\|$, we derive that

$$\begin{aligned}
\|\bar{w}^K - w^*\|^2 &= \|\bar{w}^{K-1} - w^*\|^2 - 2\eta \nabla f(\bar{w}^{K-1})^\top (\bar{w}^{K-1} - w^*) \\
&\quad + \eta^2 \|\nabla f(\bar{w}^{K-1})\|^2 \\
&\leq \|\bar{w}^{K-1} - w^*\|^2 - \eta \left(\frac{1}{\mathcal{L}} - \eta \right) \|\nabla f(\bar{w}^{K-1})\|^2 \\
&\dots \\
&\leq \|\bar{w}^0 - w^*\|^2 - \eta K \left(\frac{1}{\mathcal{L}} - \eta \right) \varepsilon^2.
\end{aligned} \tag{18}$$

Since $0 \leq f(\bar{w}^{K-1}) - f(w^*) \leq \nabla f(\bar{w}^{K-1})^\top (\bar{w}^{K-1} - w^*) - \frac{1}{2\mathcal{L}} \|\nabla f(\bar{w}^{K-1}) - \nabla f(w^*)\|^2$ ($\mathcal{L}$ -Lipschitzian), we derive that $-\nabla f(\bar{w}^{K-1})^\top (\bar{w}^{K-1} - w^*) \leq -\frac{1}{2\mathcal{L}} \|\nabla f(\bar{w}^{K-1})\|^2$ and use the result in the second step.

As a result, $\|w^K - w^*\|$ can be bounded by the sum of results in (14) and (18), i.e.,

$$\begin{aligned}
\|w^K - w^*\| &\leq \sqrt{\|\bar{w}^0 - w^*\|^2 - \eta K \left(\frac{1}{\mathcal{L}} - \eta \right) \varepsilon^2} \\
&\quad + \frac{(1 + \eta\mathcal{L})^{\tau K + \tau} - 1}{(1 + \eta\mathcal{L})^{\tau} - 1} \frac{[(1 + \eta\mathcal{L})^{\tau} - 1]\varepsilon}{M\mathcal{L}} \\
&\quad \times \sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\|.
\end{aligned} \tag{19}$$

Third, since the training dataset in FAST is a subset sampled from the entire dataset, it inevitably incurs training bias. The analysis in [19], [29], [37] shows that the gradient variance between the current model and the optimal model is inversely proportional to the size of used training data by the central limit theorem, leading to $f(w_u^K) - f(w^*) \leq \lambda \sqrt{|D_u|/|G_u|}$, where λ is a positive constant (Lemma 1 in [19]). As a result,

$$f(w^K) - f(w^*) = \frac{1}{m} \sum_{u \in \mathcal{U}} f(w_u^K) - f(w^*) \leq \frac{\lambda}{m} \sum_{u \in \mathcal{U}} \sqrt{\frac{|D_u|}{|G_u|}}. \tag{20}$$

Intuitively, a larger sampled data size implies a less loss of model accuracy.

Combining the results in (16), (17) and (20), we finally generate a new objective function as follows,

$$\sum_{u \in \mathcal{U}} \sqrt{\frac{|D_u|}{|G_u|}} \frac{\|w^K - w^*\|^2}{K}. \quad (21)$$

By minimizing (21) with the constraints in (6), the convergence bound $f(w^K) - f(w^*)$ can also be effectively minimized for obtaining a well-trained model.

B. MAB for Data Sampling

To develop a data sampling result without accessing the devices' private local data distribution, we adopt the general MAB based online learning algorithm [39], [40], [41] to adaptively determine the sampling rates. The sampled data size $|G_u|$ and data distribution Q_u are computed accordingly using (1). In FAST, the parameter server and different data sampling rates are modeled as the player and the arms in an MAB problem, respectively. Without any prior knowledge, the parameter server chooses an arm from the arm space and obtains a reward. In each training epoch, the used arms and the corresponding rewards will be adopted to guide the following arm selection operation. Specifically, the arm space and reward are defined as follows. 1) For the arm space, the MAB algorithm is traditionally developed for solving problems with discrete decision space. However, the value range of data sampling rate is a continuous space (i.e., $\bar{q}_u^c \in [0, 1]$) so that the arm space is infinite. Thus, we will partition the continuous decision space into discrete arm sets using the widely adopted approach [42], [43]. Specifically, we divide the continuous space $[0, 1]$ into a set $A = \{a_1, a_2, \dots, a_L\}$ with a finite number of values, where $a_l \in [0, 1], \forall l \in \{1, \dots, L\}$. Then, the MAB agent selects the optimal arm from the set A . 2) For the reward, we define the reward R in each training epoch k based on (21) as,

$$R(a^k) = \Delta\psi^k, \quad (22)$$

where a^k is the selected arm in epoch k , $\Delta\psi^k = \psi^{k-1} - \psi^k$ and $\psi^k = \sum_{u \in \mathcal{U}} \sqrt{\frac{|D_u|}{|G_u|}} \frac{\|w^K - w^*\|^2}{K}$. Note that the values of K , and $\|w^K - w^*\|^2$ are associated with $\bar{q}_u^c$, and the estimation of those values are classified in Section IV-C. The reward $R(a^k)$ indicates the contribution of arm a^k to the model convergence. The larger the reward, the better the performance improvement, i.e., we find better sampling rate that helps to minimize the objective (i.e., loss) of the problem in (6). The goal of the MAB is to maximize the total received rewards via a judicious trade-off between the exploration of arms that perform well in the past and the exploitation of arms that might provide higher rewards in the future, i.e., MAB will adjust its exploration-exploitation strategy based on current training performance on each device dynamically.

1) *Exploitation*. Based on the previous work [43], we define the empirical reward of each arm $a_l \in A$ in each epoch k as

$$\bar{R}^k(a_l) = \frac{\sum_{i=1}^{k-1} \theta^{k-i} \mathbb{I}_{a_l=a^k} R(a^k)}{\sum_{i=1}^{k-1} \theta^{k-i} \mathbb{I}_{a_l=a^k}} \quad (23)$$

Algorithm 1: Learning Procedure of the MAB Agent.

- 1: Initialize the arm space A , and the probability $\bar{p}^k(a_l) = \frac{1}{L}, \forall l$;
 - 2: **for** epoch $k \in \{1, 2, \dots, K\}$ **do**
 - 3: Select an arm a^k based on the probability $\bar{p}^k(a_l), \forall a_l$;
 - 4: Receive the reward $R(a^k)$ based on (22);
 - 5: Update estimated rewards $\bar{R}^k(a_l), \forall a_l$ by (23);
 - 6: Calculate the probability $\bar{p}^k(a_l), \forall a_l$ by (24);
 - 7: **end for**
-

where $\theta \in (0, 1)$ is a discount factor, i.e., the recent rewards are assigned with higher weights. The arm a_l with higher $\bar{R}^k(a_l)$ are more possible to be chosen in epoch k . That is, the agent recognizes local distributions of devices and selects arms for each of them, making the information in each class can be learned effectively.

2) *Exploration*: We further adopt the Boltzmann exploration strategy [44] to balance exploration and exploitation. That is, the probability $\bar{p}^k(a_l)$ of selecting arm a_l at epoch k is given by

$$\bar{p}^k(a_l) = \frac{e^{\gamma \bar{R}^k(a_l)}}{\sum_{l'=1}^L e^{\gamma \bar{R}^k(a_{l'})}} \quad (24)$$

where γ is the hyperparameter for balancing exploration and exploitation. Specifically, all arms are selected uniformly regardless of training epochs when $\gamma = 0$ (i.e., exploration-only). When $\gamma \rightarrow \infty$, the arm with the highest reward will be always selected (i.e., exploitation-only). For clear illustration, we conclude the above procedure of an agent in Alg. 1.

C. Deterministic Algorithm for Local Training

Given the data sampling result in each epoch k , we are able to obtain the corresponding optimal value of τ that minimizes the objective function in (21). We first define a function $g(\tau, K)$ that associates with τ and K by plugging the result of (19) to (21),

$$\begin{aligned} g(\tau, K) = & \sum_{u \in \mathcal{U}} \sqrt{\frac{|D_u|}{|G_u|}} \frac{1}{K} \left\{ \sqrt{\|w^0 - w^*\|^2 - \eta K \left(\frac{1}{\mathcal{L}} - \eta \right) \varepsilon^2} \right. \\ & + \frac{(1 + \eta \mathcal{L})^{\tau K + \tau} - 1}{(1 + \eta \mathcal{L})^\tau - 1} \frac{[(1 + \eta \mathcal{L})^\tau - 1] \varepsilon}{M \mathcal{L}} \\ & \left. \times \sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\| \right\}^2 \end{aligned} \quad (25)$$

The formula $\sum_{u \in \mathcal{U}} \sqrt{\frac{|D_u|}{|G_u|}}$ can be removed from (25) without affecting the optimal value of τ . For the value of K in epoch $k + 1$ within time constraint, it can be estimated using the first and second inequalities in (6), i.e.,

$$K = k + \frac{T - \sum_{i=1}^k t^i}{\max_{u \in \mathcal{U}} \{\tau |G_u| t_{u,cp}^k + t_{u,cm}^k\}}. \quad (26)$$

We estimate some hyperparameters based on some previous works [9], [29]. Specifically, for $A_4 = \sum_{u \in \mathcal{U}} \sum_{c=1}^C \|q_u^c - p^c\|$,

since we aim to develop the training strategy without accessing the local data distribution of each device, the value of A_4 cannot be obtained directly. Therefore, we estimate it using the current model w^k in each epoch k by (8) and (19). For the other parameters (i.e., η , $\mathcal{L}$, $\|w^0 - w^*\|^2$, and ε), η is a predefined hyperparameter (e.g., 0.01) for gradient descent, $\mathcal{L}$ and $\|w^0 - w^*\|^2$ are computed by (8) using current model and gradient informations (e.g., $\mathcal{L} = \frac{\|\nabla f_u(w_1) - \nabla f_u(w_2)\|}{\|w_1 - w_2\|}$), and ε is the largest loss over all classes. Note that the values of those estimated parameters will be updated in each epoch, and the function g is only related to the variable τ .

There are two ways to obtain the optimal value of τ that minimizes g . 1) We can adopt some automatic derivation tool (e.g., Wolfram|alpha [45]) for computing the derivative $\frac{dg}{d\tau}$ and thus obtain the optimal τ . To safely perform the derivative operation, we demand to remove the max function in (26). Specifically, we can randomly select a device and use its training speed to calculate K . This is reasonable because our optimization of data sampling and local training will potentially balance the training speed of all devices for achieving training performance improvement. 2) There also exists a more straightforward way to calculate τ , and we adopt it in the experiment. Generally, τ is a predefined parameter with limited ranges [9], [10], [46]. Thus, we can directly search for τ from 1 to τ_{max} , and adopt the one that is with the minimum $g(\tau)$. Since such operation is conducted on the parameter server with sufficient computing resource and the search space is very limited, the corresponding time consumption can be ignored.

V. PERFORMANCE EVALUATION

A. Experimental Setup

In this section, we conduct abundant experiments to evaluate the performance of our proposed framework. All experiments are conducted using a commonly used deep learning library [47], i.e., Pytorch 1.11.0 with Python 3.8.12.

Models and Datasets: The experiments are conducted on three well-known DNNs, i.e., LeNet-5 [48], AlexNet [49], and ResNet-18 [50] on three real datasets, i.e., EMNIST [51], CIFAR-10 [52], and CIFAR-100 [52]. These represent a large variety of the small, medium and large DNNs and datasets in edge computing. Specifically, the size of those DNNs is 2.021Mb, 109.09Mb, and 342.59Mb, respectively.

For the three datasets, 1) AlexNet is trained on CIFAR-10, which includes 50,000 images for training and 10,000 for testing with 10 different classes. The images in CIFAR-10 are $32 \times 32 \times 3$ dimensional. 2) CIFAR-100 has the same number of images as CIFAR-10 but consists of 100 classes, which is more challenging to train models for classification. Thus, we train the largest model (i.e., ResNet-18) on CIFAR-100. 3) The third one is EMNIST, which has much more data samples, compared to the other two datasets. EMNIST includes 814,255 images in 62 classes, of which 731,668 are for training and 82,587 are for testing. LeNet-5 is trained on EMNIST, and the dimension of each image is 28×28 pixel gray-scale.

TABLE I
PARAMETER SETUP WITH RESPECT TO NON-IID

Datasets	D	C	α			σ			χ			μ		
CIFAR-10	50,000	10	0.3	0.5	0.7	100	300	500	3	5	7	0.1	1	10
CIFAR-100	50,000	100	0.3	0.5	0.7	100	300	500	30	50	70	0.1	1	10
EMNIST	731,668	62	0.1	0.3	0.5	3,000	5,000	7,000	20	35	50	0.1	1	10

Data Distribution at Devices: We consider five different ways of distributing the datasets into devices, including one IID setting and four Non-IID settings. For the IID setting, the dataset is distributed to each device randomly and uniformly. By the comprehensive summary on Non-IID settings from the data distribution perspective in previous works [11], [14], [53], [54], we adopt the following setup. 1) *Label skew*. The devices' data distributions are different. Specifically, for each class, α of its data samples are assigned to one device, and the remaining $1 - \alpha$ are distributed to all other devices uniformly. 2) *Quantity skew*. The size of local dataset varies across devices, while their local data distributions may still be consistent. The dataset with a total size of D is distributed to m devices according to the Gaussian distribution with the same expectation (e.g., $\frac{D}{m}$) but different standard deviation σ , which denotes the data size of each device mainly ranging from $\frac{D}{m} - 3\sigma$ to $\frac{D}{m} + 3\sigma$ with a probability of 99.73%. 3) *Label-quantity skew*. Here each device owns data samples of a fixed number of classes (i.e., χ), rather than all classes. Specifically, we first assign each class to each device randomly until the number of classes reaches the budget. Then, for each class, we distribute the corresponding data samples into the devices own such class uniformly. 4) *Dirichlet distribution*. We finally use Dirichlet distribution [55], i.e., $DIR(\mu)$, to simulate real-world Non-IID data distribution. $DIR(\mu)$ is widely used as prior distribution in Bayesian statistics, and the parameter μ is adopted to control the degree of Non-IID. A smaller μ means a higher data heterogeneity.

For clear illustration, the examples of different data distribution ways are shown in Fig. 3. To effectively study how different data distribution ways influence the training performance of FAST, we take multiple values of those parameters (i.e., α , σ , χ and μ) for each dataset (i.e., CIFAR-10, CIFAR-100 and EMNIST), as shown in Table I.

Experimental Environment: Experiments are conducted on both physical platform and simulated environment. The physical environment allows us to validate the practical applicability and effectiveness of our proposed approach, while the simulated environment offers flexibility and scalability for testing and experimentation.

1) *Physical platform*. The deployment of FAST spans two parts: i) *The parameter server*. We deploy it on an AMAX deep learning workstation with an 8-core Intel(R) Xeon(R) CPU (E5-2620v4) and 4 NVIDIA GeForce RTX 2080Ti GPUs. ii) *Devices*. We use 30 devices, including 15 NVIDIA Jetson TX2, 10 NVIDIA Jetson Xavier NX and 5 NVIDIA Jetson AGX, labeled from 1 to 30. Each TX2 has one GPU and one CPU cluster (i.e., a 2-core Denver2 and a 4-core ARM CortexA57). Each NX is equipped with a 6-core NVIDIA Carmel ARMv8.2 CPU and a 384-core NVIDIA Volta GPU. The AGX carries a

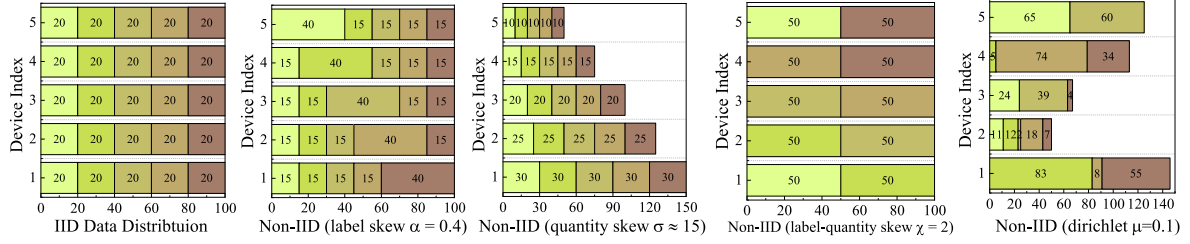


Fig. 3. Examples of different data distribution ways. There are 5 devices and 5 classes with 100 data points in each class, and the number marked in each bar denotes the number of data samples in this class.

TABLE II
COMPUTING MODES (GHZ) OF TX2, NX AND AGX

Device	Mode	Denver2	ARMA57	ARMv8.2	GPU	Numbers
TX2	1	2.0×2	2.0×4	-	1.3	1-5
	2	1.4×2	1.4×4	-	1.12	6-10
	3	-	1.2×4	-	0.85	11-15
NX	1	-	-	1.4×6	1.1	16-20
	2	-	-	1.2×4	0.8	21-25
AGX	1	-	-	1.2×4	0.67	26-28
	2	-	-	1.2×8	0.9	29-30

512-core NVIDIA Volta GPU and an 8-core NVIDIA Carmel ARMv8.2 CPU. The following settings are further adopted to reflect the device heterogeneity. For computing heterogeneity, we take different computing modes for each type of devices, as shown in Table II. For communication heterogeneity, all devices are arranged at different places in an office. Since the bandwidth of one single router is occupied by 30 devices, the transmission rate (measured by iperf3 [56]) between devices and the parameter server during training dynamically fluctuates between 1.6 Mbps and 14 Mbps.

2) Simulated environment. We simulate an edge computing scenario with 1 parameter server and 30 devices in a workstation with 2 40-core Intel(R) Xeon(R) Gold 5218R CPU@2.10 GHz and 8 NVIDIA GeForce RTX 3090 GPUs. According to the tests on the physical devices (e.g., TX2/NX), we adopt the following configurations. 1) *Communication*. To simulate the heterogeneous and dynamic network conditions, we let the bandwidth between devices and parameter servers fluctuate between 1 Mbps to 10 Mbps, and randomly change the bandwidth every 10 epochs. 2) *Computing*. We also allocate different computing capacities to each virtual device. Specifically, for AlexNet on CIFAR-10, the measured computing cost on TX2/NX varies between 0.8 s to 5.2 s for processing 100 data points. The corresponding time consumption fluctuates between 1.8 s to 8.5 s for ResNet on CIFAR-100, and varies between 0.15 s to 0.75 s for LeNet-5 on EMNIST. We randomly assign the computing time with such ranges for different devices when performing the corresponding learning tasks, and dynamically adjust the time every 10 epochs.

Baselines: We mainly evaluate the efficiency of FAST by comparing it with four baselines. 1) Centralized training (CT). All devices share and transmit their local dataset to a central node for training. Since centralized training ignores the privacy issue and resource constraints on devices, we only regard it as

an upper bound for the convergence performance comparison. 2) CSFedAvg [27] utilizes weight divergence to recognize the Non-IID degrees of devices, and the devices with lower degrees will be chosen for training. Compared with this baseline, we can verify whether FAST can effectively handle the Non-IID issue or not. 3) FLAS [33] samples data points with a loss value smaller than a threshold to balance the relationship between the personalized local models and the global model. Based on this, we can observe whether our data sampling approach outperforms current sampling technologies or not. 4) FedDistill [57] is a data-free knowledge distillation, which enables each device collectively train a generative model and shares label-wise average of logit-vectors. 5) FedGen [54] let the parameter server learn a generator to ensemble device information and then regulate their local training using the learned knowledge. 6) Rand-FAST is a modified version of FAST. The difference is that its data sampling results and the number of local iterations are randomly generated in each epoch. Furthermore, we will also take different values of the number of local iterations (from 1 to 20) and the data sampling rates (from 0 to 1) in some experiments to evaluate the performance of our proposed algorithm.

Experimental Parameters and Metrics: In all experiments, we by default set the learning rate η as 0.01 for CIFAR-10 and 0.001 for CIFAR-100/EMNIST, and the number of local iterations in each epoch as $\tau = 10$ [9]. We set the discount factor θ as 0.95 [58], the number of arms L as 20, and τ_{max} as 20 in FAST. We evaluate the performance of different frameworks by adopting three metrics as follows: 1) *Classification accuracy* is the proportion of correctly classified samples to all testing samples. 2) *Completion time* is the time duration to achieve the target test accuracy. 3) *Network traffic* is the total size of the model transmitted through the network. We adopt the average results of 3 independent experiments to avoid accidents.

B. Overall Performance With IID Data

We first present the performance of all frameworks with the IID data. Specifically, we observe how the accuracy changes with the increasing number of epochs (or time), and measure the network traffic consumption. The total time budget until training terminates is by default set as 100,000 s for LeNet and AlexNet, and 150,000 s for ResNet-18, respectively. The results are shown in Figs. 4, 5, and 6 and Table III.

Convergence Performance: Figs. 4, 5, and 6 show the performance for LeNet-5 on EMNIST, AlexNet on CIFAR-10, and

TABLE III
TIME (S) AND NETWORK TRAFFIC (MB) CONSUMPTION FOR ACHIEVING THE TARGET ACCURACY

Frameworks	LeNet-5 (85%)		AlexNet (70%)		ResNet-18 (45%)	
	Time	Traffic	Time	Traffic	Time	Traffic
FAST	29,219±438	318±15	5,371±112	7,363±247	14,480±563	59,098±689
CSFedAvg	30,144±853	394±43	17,621±782	37,637±1,279	85,311±5,751	282,858±14,643
FLAS	29,634±662	455±74	11,476±436	25,364±840	20,150±2,784	89,933±2,173
Rand-FAST	31,999±1,325	182±56	21,190±1,096	17,182±959	43,876±4,524	100,211±8,530

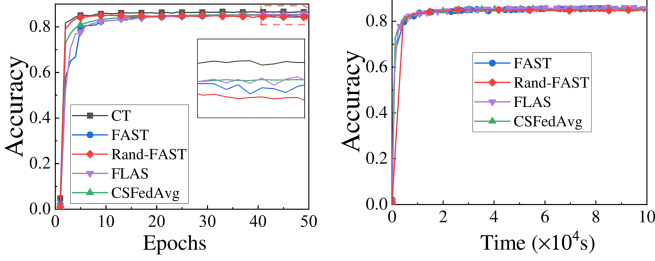


Fig. 4. Training performance for LeNet-5 on EMNIST.

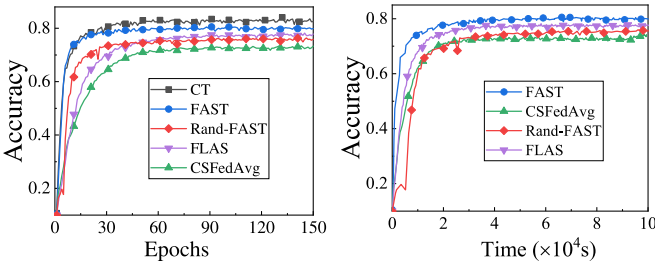


Fig. 5. Training performance for AlexNet on CIFAR-10.

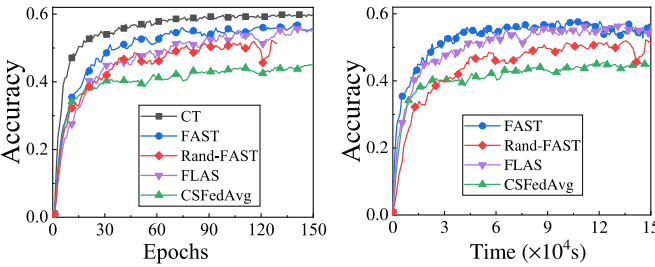


Fig. 6. Training performance for ResNet-18 on CIFAR-100.

ResNet-18 on CIFAR-100, respectively. For the small model (i.e., LeNet-5), since the data distribution among devices is IID, all frameworks achieve similar training performance. For CT, although we aim to minimize the performance gap between FAST and CT through adaptive data sampling and local training, this gap is inevitably enlarged with the increase of the model size. However, we note that FAST achieves the best training performance among all baselines except CT. For example, the accuracy of FAST, CSFedAvg, FLAS, and Rand-FAST is 78.93%, 73.32%, 77.84%, and 75.91% respectively for AlexNet on CIFAR-10 after 150 epochs.

Resource Consumption: We further compare the time and network traffic consumptions of different frameworks under the same target accuracy in Table III.

1) From the perspective of time consumption, FAST converges much faster than baselines. For AlexNet, FAST achieves a speedup of about $3.3\times$, $2.14\times$, and $3.95\times$, compared with CSFedAvg, FLAS, and Rand-FAST when achieving the same target accuracy, i.e., an accuracy that all framework can achieve. For ResNet-18, the speedup is $5.89\times$, $1.39\times$, and $3.03\times$, correspondingly. That is because FAST can effectively balance the computing and communication cost of each device by adaptively adjusting the value of data sampling rate and number of local iterations, thus significantly reducing the synchronization cost of the heterogeneous system. As a result, the training procedure will be accelerated.

2) In terms of the network traffic consumption, FL always consumes extremely large amount of traffic resource in edge computing, e.g., 10^3 - 10^6 Mb in the experiment, but Table III shows that FAST often consumes less network traffic resource than these baselines. For AlexNet on CIFAR-10, FAST reduces the consumption of network traffic by about 80.4%, 70.9%, and 57.1% compared with CSFedAvg, FLAS, and Rand-FAST. Furthermore, the larger the DNN size, the better the effect of FAST on traffic consumption reduction. Compared with FLAS, FAST saves 137 MB, 18,001 MB and 30,835 MB of traffic for LeNet-5, AlexNet and ResNet-18, respectively. By adjusting the number of local iterations among two consecutive training epochs, FAST reduces the total number of required training epochs for model convergence and thus greatly reduces the network traffic consumption.

C. Comparison Among Different Label Skew Settings

As shown in Fig. 7, we have the following observations in aspect of different frameworks under various label skew settings ($\alpha = 0$ denotes the data distribution among devices are IID). First, for different models on multiple datasets, FAST usually outperforms baselines with different label skew settings. For example, for ResNet-18 on Cifar-100 with $\alpha = 0.7$ (the hardest learning task), FAST achieves an accuracy of 43.14%, which is higher than that of CSFedAvg (32.72%), FLAS (40.21%), and Rand-FAST (39.04%). This denotes that FAST enables great potential in handling the Non-IID issue using proper adaptive data sampling and local training strategy. Second, as α increases, the training performance of all frameworks has been inevitably damaged. But FAST alleviates the impact of label skew on the performance and achieves better accuracy than baselines. For AlexNet on CIFAR-10 with $\alpha = 0.3$, the data distributions of

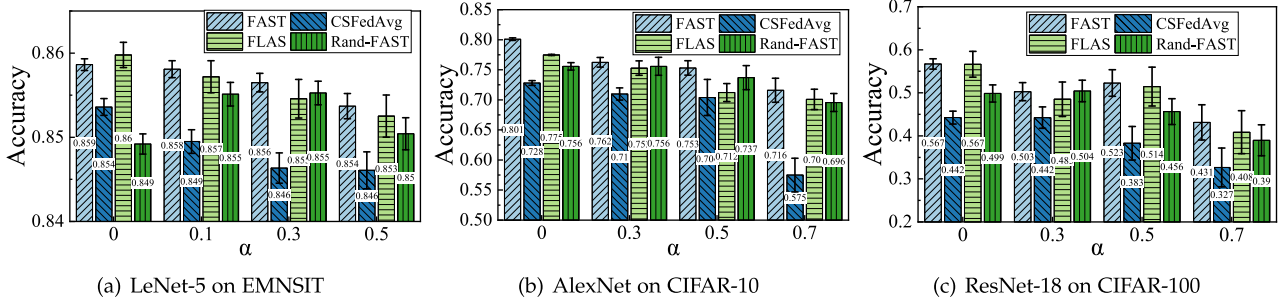


Fig. 7. Final accuracy of different frameworks with different label skew levels when the time budget is exceeded.

TABLE IV
ACCURACY, TIME (S) AND NETWORK TRAFFIC (MB) CONSUMPTIONS FOR DIFFERENT FRAMEWORKS WITH QUANTITY SKEW

Training tasks		LeNet-5 on EMNIST			AlexNet on CIFAR-10			ResNet-18 on CIFAR-100		
δ		3000	5000	7000	100	300	500	100	300	500
FAST	Accuracy	85.62±0.08	85.61±0.06	85.62±0.11	77.22±0.18	77.6±0.21	77.47±0.11	55.4±1.3	53.6±2.2	53.6±3.7
	Time	33,483±628	32,338±588	47,039±974	10,612±351	35,617±1,287	40,447±1,552	22,060±893	24,266±937	28,801±1,241
	Traffic	454±28	576±32	606±58	9,818±293	17,182±480	28,637±736	84,790±1,208	82,221±1,334	95,068±2,195
CSFedAvg	Accuracy	85.07±0.04	85.34±0.1	85.27±0.03	74.92±0.42	74.47±0.16	74.03±0.07	46.0±1.7	43.2±1.1	41.4±2.9
	Time	24,681±615	34,437±923	50,205±1,565	17,077±862	74,721±2,557	91,870±3,512	35,534±2,905	88,699±5,436	132,222±8,291
	Traffic	272±22	288±26	272±38	33,547±1,404	59,415±2,329	68,731±2,804	69,374±1,832	172,150±4,263	210,691±7,305
FLAS	Accuracy	85.79±0.1	85.65±0.19	85.51±0.12	77.14±0.1	77.41±0.26	76.38±0.45	53.1±4.2	51.2±4.8	49.2±4.6
	Time	27,169±691	36,347±825	49,066±1,251	12,159±524	20,850±737	50,711±1,963	25,944±2,404	38,167±4,845	36,976±6,334
	Traffic	439±79	424±66	485±98	24,546±738	36,002±957	39,472±1307	82,221±3,273	105,345±6,300	87,359±8,491
Rand-FAST	Accuracy	85.54±0.05	85.72±0.09	85.75±0.02	75.35±0.88	75.06±0.4	76.13±0.12	53.8±2.1	51.1±1.9	49.4±3.2
	Time	47,144±1,585	38,765±1,291	48,984±2,157	15,986±1,422	59,067±2,875	47,849±1,927	27,293±3,584	29,220±3,132	37,815±5,084
	Traffic	197±45	257±53	303±64	13,909±1,283	22,910±1,858	110,460±8,308	71,943±5,907	74,512±7,867	82,221±9,927

The optimal values for each set of experiments have been bolded.

two devices may be $\{0.55, 0.05, 0.05, \dots\}$ and $\{0.05, 0.55, 0.05, \dots\}$. The baselines usually cannot well handle such distribution divergence. However, it is not the case in FAST. FAST will sample most of data samples in the classes with fewer data points and select a small fraction of data samples in the main classes to reduce the distribution divergence among devices during per-epoch training. Besides, since FAST alleviates the synchronization cost among heterogeneous devices, it always iterates more times and then achieves a better performance under a given time budget. For example, compared with CSFedAvg, the accuracy improvement of FAST increases from 7.3% to 14.1% as α increases from 0 to 0.7. Discarding one device in CSFedAvg means that one class of data points can hardly be trained and classified (i.e., most of the data points in this class have been discarded), especially with higher label skew levels.

D. Comparison Among Different Quantity Skew Settings

As we know, the quantity skew will not incur data distribution divergence among devices, but it is also interesting to see the effect of the quantity imbalance on different FL frameworks. Table IV shows the final accuracy of all frameworks under the time budget, and compares their time and traffic consumptions given the same target accuracy requirement (i.e., the lowest final accuracy among all frameworks). In general, all frameworks achieve similar accuracy with data quantity skew, but FAST usually accelerates the training procedure to a great extent. For

AlexNet on CIFAR-10 with $\sigma = 500$, FAST consumes 40,447 s to achieve an accuracy of 74%, but the time consumption of CSFedAvg, FLAS and Rand-FAST is 91,870 s, 50,711 s, and 47,849 s, respectively, i.e., FAST is 1.2-2.3 $\times$ faster than baselines. For ResNet on CIFAR-100, the speedup is 4.6 $\times$, 1.3 $\times$, and 1.3 $\times$, correspondingly. As mentioned before, FAST can effectively handle device heterogeneity. Since quantity skew will greatly expand the heterogeneity of training speed between devices, FAST outperforms baselines, especially with higher quantity skew levels and more complex models and datasets. Specifically, when $\sigma = 500$, the dataset size among devices varies from 167 to 3,126 for CIFAR-10 and CIFAR-100, leading to a huge synchronization barrier in baselines. Undoubtedly, FAST can alleviate such conditions through data sampling.

E. Comparison Among Different Label-Quantity Skew Settings

For the label-quantity skew, since each device only owns data points in χ classes and there is no data point in the remaining classes, it brings great challenges to distributed model training. Figs. 8 and 9 plot the experimental results for LeNet-5 on EMNIST and ResNet-50 on RCIFAR-100 under different label-quantity skew settings. Two observations are as follows. First, as χ increases, the training performance of all frameworks is improved with no doubt, and FAST is more robust with less performance gap among different label-quantity skew levels. Second, as shown in the right plots of Figs. 8 and 9, we notice

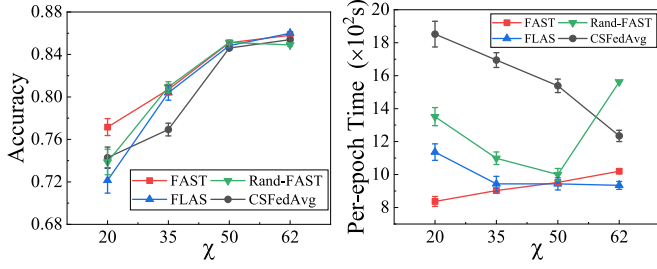


Fig. 8. Training performance for LeNet-5 on EMNIST with different label-quantity skew levels.

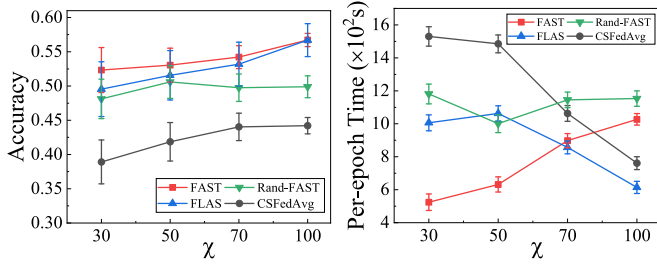


Fig. 9. Training performance for ResNet-18 on CIFAR-100 with different label-quantity skew levels.

that the per-epoch time of FAST is smaller than baselines with small χ and gradually increase as χ become larger. In fact, with label-quantity skew, the distribution divergence among devices can hardly be effectively reduced even for FAST by data sampling. This is because we are unable to sample data from those empty classes. Therefore, FAST intuitively reduces the number of local iterations between two consecutive global aggregations for avoiding the local model to deviate from the global optima, leading to a small per-epoch time. As χ increases, the number of local iterations will be increased accordingly with longer per-epoch time.

F. Effect of the Number of Local Iterations on Testbed

The above simulation results in Sections V-B to V-E verify that only by jointly optimizing data sampling and local training, FAST can achieve resource-effective and convergence-guaranteed model training under different data distribution cases. On the one hand, to dig deeper into the impact of data sampling or local training on system performance, and on the other hand, to observe the performance of our framework in practical edge computing scenario, we have further conducted experiments on testbed in Sections V-F and V-H.

We first investigate how the training performance changes with the number of training iterations τ , and Fig. 10 shows the training curves of FAST with constant τ (i.e., 1, 5, 10, 15, 20) and adaptive τ under different data distribution settings. We find that the value of τ can have a large effect on the accuracy of FAST, and its optimal value is sensitive to different Non-IID distributions. A small τ (e.g., 1) always incurs a lot of communication cost due to its frequent global aggregation, resulting in a slow convergence rate. However, without considering the time budget, this approach will eventually achieve the optimal

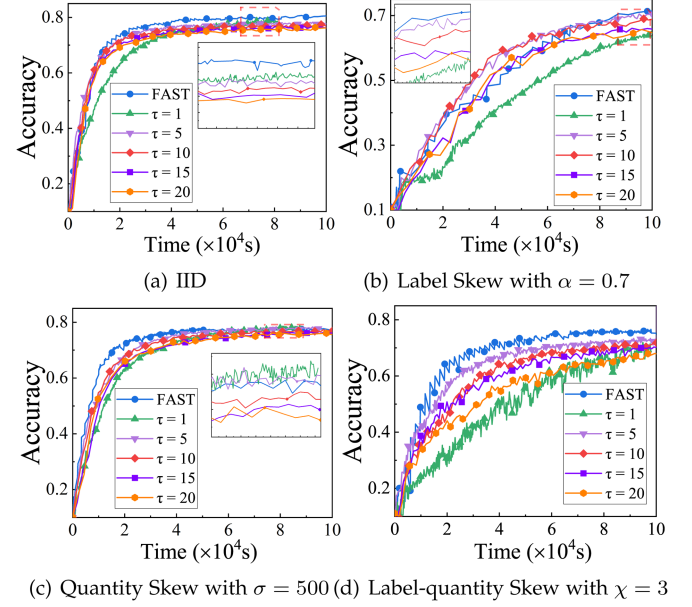


Fig. 10. Training performance of FAST with different number of local iterations under different data distributions.

accuracy. Besides, as τ increases (e.g., 5, 10, 15, 20), the final accuracy gradually decreases regardless of the data distribution settings, which is consistent with the convergence analysis in Section III. For FAST with adaptive τ , we notice that it prefers to select small τ in the early stage of model training, and adopt large τ when the convergence rate tends to be slow and stable. Such adaptive strategy can both make the full utilization of the resource and achieve good convergence performance. Specifically, for the label-quantity skew with $\chi = 3$, the accuracy for FAST with different τ (i.e., 1, 5, 10, 15, 20 and adaptive) is 71.43%, 73.27%, 71.23%, 70.13%, 68.04% and 74.82%, and the adaptive strategy obtains an accuracy improvement of about 1.55-6.78%.

G. Effect of the Data Sampling Rate on Testbed

As we know, the sampling rate of each class on each device always ranges from 0 to 1. Using the largest search space [0, 1] ensures that MAB agent will not miss the optimal sampling rate, but it also brings more search cost. Thus, in the last set of experiments, we analyze the impacts of the search space of the sampling rate on the training performance. We make two observations from the results in Table V. First, when the local data distribution is consistent among devices (i.e., IID and quantity skew), all search space obtains similar final accuracy. However, a large search space enables FAST to reduce the synchronization cost as much as possible and thus achieve a faster convergence rate. Second, given different data distributions among devices, as the sampling rate gets larger, the accuracy usually decreases. But the optimal search space lies on [0.2, 1]. For label skew with $\alpha = 0.7$, the data distribution of one device may be {0.85, 0.017, 0.017, ...}. As a result, the MAB agent will adopt a relatively small sampling rate of a certain class for reducing the distribution divergence, and such class may not be fully utilized

TABLE V
FINAL ACCURACY WITH DIFFERENT SEARCH SPACES OF THE SAMPLING RATE

Search Space	IID	Label Skew with $\alpha = 0.7$	Quantity Skew with $\sigma = 500$	Label-quantity Skew with $\chi = 3$
[0.0,1.0]	80.1%	71.54%	77.68%	74.81%
[0.2,1.0]	79.67%	72.73%	77.11%	75.25%
[0.4,1.0]	79.42%	70.37%	77.22%	74.92%
[0.6,1.0]	79.84%	69.65%	76.52%	74.55%
[0.8,1.0]	78.94%	67.87%	76.34%	73.97%
[1.0,1.0]	79.91%	68.04%	76.44%	72.85%

The optimal values for each set of experiments have been bolded.

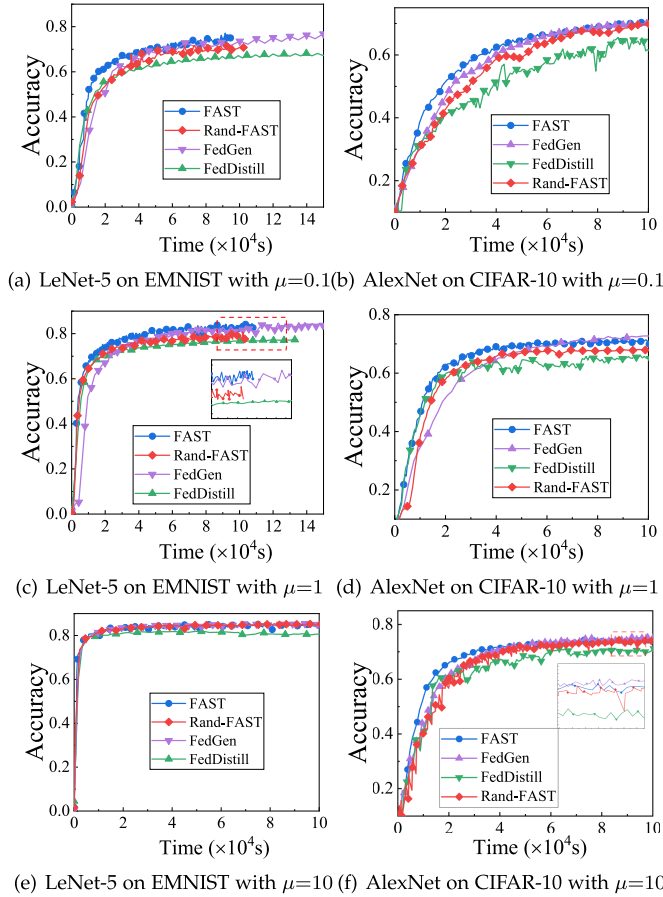


Fig. 11. Training performance for LeNet-5 on EMNIST and AlexNet on CIFAR-10 with different Dirichlet distributions.

during multiple training iterations. At this moment, a lower bound of the sampling rate (e.g., 0.2) contributes to performance improvement. However, the optimal search space may vary with training tasks and data distributions, and the approach to obtain its optimal value demands further study.

H. Comparison Among Different Dirichlet Distribution Settings on Testbed

In this set of experiments, we have compared the training performance of FAST with baselines under different Dirichlet distributions, and Fig. 11 shows the result for LeNet-5 on EMNIST and AlexNet on CIFAR-10. We have three observations. First, we note that FAST converges faster than baselines under

different levels of data heterogeneity. For example, to achieve the final accuracy (67.2%) of FedDistill for LeNet-5 on EMNIST after 150,000 s, the time consumption of FAST, Rand-FAST, and FedGen is 33,051 s, 53,074 s and 62,195 s, respectively. Both FedGen and FedDistill generate pseudo-data to extract knowledge by knowledge distillation to regulate local training of devices, and they require additional heavy computation and communication for training and uploading the generator model besides the original model. But for FAST, by jointly optimizing the data sampling and local training, FAST can improve the efficiency of resource utilization and enhance the performance of model training simultaneously. Second, similar to the experiment results in Figs. 7, 8, and 9, the training performance of all approaches is gradually getting better as the level of data heterogeneity among devices decreases (i.e., as μ increases). Third, for FedGen, we note that it may obtain a slightly higher final accuracy than FAST. The core idea of both FedGen and FAST is to extract knowledge about the global view of data distribution to guide the local learning process of devices. The difference is that FAST enables each device to sample different rates of data points from each of its local classes to rebuild a dataset with a distribution closer to the global distribution, while FedGen lets devices use additionally trained generator model to generate some pseudo-data to make the local data distribution closer to the global distribution. As a result, the extra resource overhead is traded for possible performance improvement. However, for FedGen, the quality of generated samples is sensitive to the generator model as well as many hyperparameters involved in the training procedure, and generating inappropriate data may further damage model training. For FedDistill, it shares a label-wise average of logits among devices rather than trained local models. Since FedDistill has to extract knowledge from these logits for parameter updating rather than adopting direct model parameter aggregation, it experiences non-negligible performance drops compared with other baselines.

VI. RELATED WORKS

FL has emerged as a distributed AI approach that enables devices collaboratively to solve a learning problem [59], [60]. Many previous works [21], [29] show that the performance degradation always gets larger as data skewness and heterogeneity among devices increases. To handle those issues, existing frameworks can be roughly divided into two categories, i.e., the data-centric and model-centric methods.

A. Data-Centric Method

This method aims to adjust the data (distribution) trained over the entire system and then improve the accuracy of the averaged global model, which is the main method discussed in this paper. Zhao et al. [13] quantify the data distribution difference among devices by earth mover's distance and propose a cross-device data sharing approach to improve the training performance. Since data sharing may incur additional communication overhead and privacy leakage, many authors [3], [14], [27], [61], [62], [63] investigate various device sampling strategies. Wang et al. [14] develop an experience-driven control framework for device sampling to counterbalance the bias induced by Non-IID

data. The authors in [63] measure the difference between the global and local models using the proposed normalized model divergence, and then exclude the outlier updates to improve the training efficiency. As a more fine-grained sampling approach, the sample-level data sampling has been studied recently. Feng et al. [30] enable each device to train a randomly selected subset of data points to alleviate the computing cost and accelerate training procedure. Considering the loss distribution among devices, Shu et al. [33] design an adaptive data sampling method for each device to select the training data, ensuring that local models converge consistently.

B. Model-Centric Method

Such method targets to adopt some advanced techniques to process the model trained by each device. FedProx [8] adds a proximal term to the objective, which provides a principled way for the parameter server to account for different data distributions and helps to improve the convergence stability. FedUFO [64] designs a group and a global consensus loss function to explicitly align the local, group and global optimization objectives to alleviate the Non-IID issue in FL. Instead of averaging the local models of devices equally by the parameter server, some authors [15], [17], [65] adopt different model averaging approaches by deeply exploring model architecture. For example, FedBN [17] shows that keeping the BN layers of each device updated locally without any communication and aggregation, and only averaging the non-BN layers can address the Non-IID issue. The authors in [65] propose a distributionally averaging algorithm that employs a snapshotting scheme to approximate the accumulation of history gradients of the global model. Some recent works [54], [57] generate pseudo-data to extract knowledge by knowledge distillation to regulate local training, they require additional heavy computation and communication for training and uploading the generator model, and the quality of samples is sensitive to the many hyperparameters involved in the training procedure.

In contrast to the above researches, our work addresses the resource constraints and Non-IID issue by adaptive data sampling and local training. Apparently, FAST belongs to the data-centric method, and partial model-centric methods are orthogonal to our work and can be used in conjunction to further improve the training performance of FAST.

VII. CONCLUSION

In this paper, we have proposed an efficient federated learning framework, FAST, to enhance federated learning by conquering some critical issues, including resource constraints, system heterogeneity and Non-IID data distribution, in edge computing. Specifically, we have formulated an optimization problem aiming to minimize the global loss under a given time budget. Guided by the convergence analysis, we have designed the online learning algorithm to adaptively develop the data sampling and local training strategies so as to improve the training performance without accessing devices' private data distributions. The

experimental results show that FAST significantly outperforms the existing FL frameworks.

REFERENCES

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [2] Z. Wang, H. Xu, J. Liu, H. Huang, C. Qiao, and Y. Zhao, "Resource-efficient federated learning with hierarchical aggregation in edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [3] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *Proc. IEEE Int. Conf. Commun.*, 2019, pp. 1–7.
- [4] W. Y. B. Lim et al., "Decentralized edge intelligence: A dynamic resource allocation framework for hierarchical federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 536–550, Mar. 2022.
- [5] Q. Yang, Y. Liu, Y. Cheng, Y. Kang, T. Chen, and H. Yu, "Federated learning," *Synth. Lectures Artif. Intell. Mach. Learn.*, vol. 13, no. 3, pp. 1–207, 2019.
- [6] M. Duan et al., "Flexible clustered federated learning for client-level data distribution shift," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2661–2674, Nov. 2022.
- [7] K. Bonawitz et al., "Towards federated learning at scale: System design," *Proc. Mach. Learn. Syst.*, vol. 1, pp. 374–388, 2019.
- [8] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, May 2020.
- [9] S. Wang et al., "When edge meets learning: Adaptive control for resource-constrained distributed machine learning," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 63–71.
- [10] C. Xie, S. Koyejo, and I. Gupta, "Asynchronous federated optimization," 2019, *arXiv:1903.03934*.
- [11] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on Non-IID data silos: An experimental study," 2021, *arXiv:2102.02079*.
- [12] M. Duan, D. Liu, X. Chen, R. Liu, Y. Tan, and L. Liang, "Self-balancing federated learning with global imbalanced data in mobile systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 59–71, Jan. 2021.
- [13] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with Non-IID data," 2018, *arXiv:1806.00582*.
- [14] H. Wang, Z. Kaplan, D. Niu, and B. Li, "Optimizing federated learning on Non-IID data with reinforcement learning," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 1698–1707.
- [15] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 7611–7623, 2020.
- [16] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 450–465, Feb. 2018.
- [17] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "FedBN: Federated learning on Non-IID features via local batch normalization," 2021, *arXiv:2102.07623*.
- [18] R. Yonetani, T. Takahashi, A. Hashimoto, and Y. Ushiku, "Decentralized learning of generative adversarial networks from multi-client Non-IID data," 2019, *arXiv:1905.09684*.
- [19] Y. Tu, Y. Ruan, S. Wagle, C. G. Brinton, and C. Joe-Wong, "Network-aware optimization of distributed learning for fog computing," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 2509–2518.
- [20] J. M. M. Rumbold and B. Pierscionek, "The effect of the general data protection regulation on medical research," *J. Med. Internet Res.*, vol. 19, no. 2, 2017, Art. no. e7108.
- [21] M. Chen, N. Shlezinger, H. V. Poor, Y. C. Eldar, and S. Cui, "Communication-efficient federated learning," *Proc. Nat. Acad. Sci. USA*, vol. 118, no. 17, 2021, Art. no. e2024789118.
- [22] P. Kairouz et al., "Advances and open problems in federated learning," 2019, *arXiv:1912.04977*.
- [23] Y. J. Cho, J. Wang, and G. Joshi, "Client selection in federated learning: Convergence analysis and power-of-choice selection strategies," 2020, *arXiv:2010.01243*.
- [24] J. Perazzone, S. Wang, M. Ji, and K. Chan, "Communication-efficient device scheduling for federated learning using stochastic optimization," 2022, *arXiv:2201.07912*.
- [25] B. Luo, W. Xiao, S. Wang, J. Huang, and L. Tassiulas, "Tackling system and statistical heterogeneity for federated learning with adaptive client sampling," 2021, *arXiv:2112.11256*.

- [26] Y. Jin, L. Jiao, Z. Qian, S. Zhang, S. Lu, and X. Wang, "Resource-efficient and convergence-preserving online participant selection in federated learning," in *Proc. IEEE 40th Int. Conf. Distrib. Comput. Syst.*, 2020, pp. 606–616.
- [27] W. Zhang, X. Wang, P. Zhou, W. Wu, and X. Zhang, "Client selection for federated learning with Non-IID data in mobile edge computing," *IEEE Access*, vol. 9, pp. 24 462–24 474, 2021.
- [28] C. Chen, W. Wang, and B. Li, "Round-robin synchronization: Mitigating communication bottlenecks in parameter servers," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 532–540.
- [29] Z. Ma, Y. Xu, H. Xu, Z. Meng, L. Huang, and Y. Xue, "Adaptive batch size for federated learning in resource-constrained edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 37–53, Jan. 2023.
- [30] C. Feng, Y. Wang, Z. Zhao, T. Q. Quek, and M. Peng, "Joint optimization of data sampling and user selection for federated learning in the mobile edge computing systems," in *Proc. IEEE Int. Conf. Commun. Workshops*, 2020, pp. 1–6.
- [31] T. Tuor, S. Wang, B. J. Ko, C. Liu, and K. K. Leung, "Data selection for federated learning with relevant and irrelevant data at clients," 2020, *arXiv:2001.08300*.
- [32] H. Shi and V. Radu, "Data selection for efficient model update in federated learning," in *Proc. 2nd Eur. Workshop Mach. Learn. Syst.*, 2022, pp. 72–78.
- [33] J. Shu, W. Zhang, Y. Zhou, Z. Cheng, and L. T. Yang, "FLAS: Computation and communication efficient federated learning via adaptive sampling," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2003–2014, Jul./Aug. 2022.
- [34] A. Li, L. Zhang, J. Tan, Y. Qin, J. Wang, and X.-Y. Li, "Sample-level data selection for federated learning," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [35] Z. Zhang and M. Sabuncu, "Generalized cross entropy loss for training deep neural networks with noisy labels," *Adv. Neural Inf. Process. Syst.*, vol. 31, pp. 8792–8802, 2018.
- [36] P. Goyal et al., "Accurate, large minibatch SGD: Training imagenet in 1 hour," 2017, *arXiv: 1706.02677*.
- [37] J. Zhang et al., "Adaptive federated learning on Non-IID data with resource constraint," *IEEE Trans. Comput.*, vol. 71, no. 7, pp. 1655–1667, Jul. 2022.
- [38] N. Mhaisen, A. Awad, A. Mohamed, A. Erbad, and M. Guizani, "Optimal user-edge assignment in hierarchical federated learning based on statistical properties and network topology constraints," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 1, pp. 55–66, Jan./Feb. 2022.
- [39] W. Xia, T. Q. Quek, K. Guo, W. Wen, H. H. Yang, and H. Zhu, "Multi-armed bandit-based client scheduling for federated learning," *IEEE Trans. Wireless Commun.*, vol. 19, no. 11, pp. 7108–7123, Nov. 2020.
- [40] B. Xu, W. Xia, J. Zhang, T. Q. Quek, and H. Zhu, "Online client scheduling for fast federated learning," *IEEE Wireless Commun. Lett.*, vol. 10, no. 7, pp. 1434–1438, Jul. 2021.
- [41] D. Qiao, S. Guo, D. Liu, S. Long, P. Zhou, and Z. Li, "Adaptive federated deep reinforcement learning for proactive content caching in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4767–4782, Dec. 2022.
- [42] J. Vermorel and M. Mohri, "Multi-armed bandit algorithms and empirical evaluation," in *Proc. Eur. Conf. Mach. Learn.*, Springer, 2005, pp. 437–448.
- [43] A. Krishnamurthy, J. Langford, A. Slivkins, and C. Zhang, "Contextual bandits with continuous actions: Smoothing, zooming, and adapting," in *Proc. Conf. Learn. Theory*, PMLR, 2019, pp. 2025–2027.
- [44] N. Cesa-Bianchi, C. Gentile, G. Lugosi, and G. Neu, "Boltzmann exploration done right," *Adv. Neural Inf. Process. Syst.*, vol. 30, pp. 6287–6296, 2017.
- [45] V. E. Dimiceli, A. S. Lang, and L. Locke, "Teaching calculus with wolfram alpha," *Int. J. Math. Educ. Sci. Technol.*, vol. 41, no. 8, pp. 1061–1071, 2010.
- [46] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *Proc. 2020 IEEE Int. Conf. Commun.*, 2020, pp. 1–6.
- [47] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," *Adv. Neural Inf. Process. Syst.*, vol. 32, pp. 8026–8037, 2019.
- [48] Y. LeCun et al., "LeNet-5, convolutional neural networks," 2015. [Online]. Available: <http://yann.lecun.com/exdb/lenet>
- [49] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Adv. Neural Inf. Process. Syst.*, vol. 25, pp. 1097–1105, 2012.
- [50] S. Targ, D. Almeida, and K. Lyman, "Resnet in ResNet: Generalizing residual architectures," 2016, *arXiv:1603.08029*.
- [51] G. Cohen, S. Afshar, J. Tapson, and A. Van Schaik, "EMNIST: Extending MNIST to handwritten letters," in *Proc. Int. Joint Conf. Neural Netw.*, 2017, pp. 2921–2926.
- [52] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," Toronto, ON, Canada, Tech. Rep., 2009.
- [53] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 2351–2363, 2020.
- [54] Z. Zhu, J. Hong, and J. Zhou, "Data-free knowledge distillation for heterogeneous federated learning," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 12 878–12 889.
- [55] T. Minka, "Estimating a dirichlet distribution," MIT, Tech. Rep., 2000.
- [56] A. Tirumala, "iPerf: The TCP/UDP bandwidth measurement tool," 1999. [Online]. Available: <http://dast.nlanr.net/Projects/Iperf/>
- [57] E. Jeong, S. Oh, H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Communication-efficient on-device machine learning: Federated distillation and augmentation under Non-IID private data," 2018, *arXiv: 1811.11479*.
- [58] A. Garivier and E. Moulines, "On upper-confidence bound policies for non-stationary bandit problems," 2008, *arXiv:0805.3415*.
- [59] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Federated learning for Internet of Things: A comprehensive survey," *IEEE Commun. Surv. Tut.*, vol. 23, no. 3, pp. 1622–1658, Third Quarter 2021.
- [60] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, PMLR, 2017, pp. 1273–1282.
- [61] M. Ribero and H. Vikalo, "Communication-efficient federated learning via optimal client sampling," 2020, *arXiv: 2007.15197*.
- [62] W. Chen, S. Horvath, and P. Richtarik, "Optimal client sampling for federated learning," 2020, *arXiv: 2010.13723*.
- [63] W. Luping, W. Wei, and L. Bo, "CMFL: Mitigating communication overhead for federated learning," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 954–964.
- [64] L. Zhang, Y. Luo, Y. Bai, B. Du, and L.-Y. Duan, "Federated learning for Non-IID data via unified feature learning and optimization objective alignment," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 4420–4428.
- [65] Y. Deng, M. M. Kamani, and M. Mahdavi, "Distributionally robust federated averaging," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 15 111–15 122, 2020.



Zhiyuan Wang received the BS degree from the Jilin University in 2019. He is currently working toward the PhD degree with the School of Computer Science, University of Science and Technology of China (USTC). His main research interests are edge computing, federated learning and distributed machine learning.



Hongli Xu (Member, IEEE) received the BS degree in computer science from the University of Science and Technology of China, China, in 2002, and the PhD degree in computer software and theory from the University of Science and Technology of China, China, in 2007. He is a professor with the School of Computer Science and Technology, University of Science and Technology of China, China. He was awarded the Outstanding Youth Science Foundation of NSFC, in 2018. He has won the best paper award or the best paper candidate in several famous conferences. He has published more than 100 papers in famous journals and conferences, including the *IEEE/ACM Transactions on Networking*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *Infocom* and *ICNP*, etc. He has also held more than 30 patents. His main research interest is software defined networks, edge computing and Internet of Thing.



Yang Xu (Member, IEEE) received the BS degree from the Wuhan University of Technology in 2014 and the PhD degree in computer science and technology from University of Science and Technology of China in 2019. He is currently an associate researcher with the School of Computer Science and Technology at University of Science and Technology of China. His research interests include ubiquitous computing, deep learning and mobile edge computing.



Jianchun Liu (Member, IEEE) received the PhD degree from the School of Data Science from the University of Science and Technology of China in 2022. He is currently an associate researcher with the School of Computer Science and Technology, University of Science and Technology of China. His main research interests are software defined networks, network function virtualization, edge computing and federated learning.



Zhida Jiang received the BS degree from the Hefei University of Technology in 2019. He is currently working toward the PhD degree with the School of Computer Science and Technology, University of Science and Technology of China (USTC). His research interests include edge computing and federated learning.



Suo Chen received the BS degree in information and computing sciences from the Nanjing University of Aeronautics and Astronautics, China, in 2018. He is currently working toward the PhD degree with the School of Computer Science and Technology, University of Science and Technology of China. His research interests include edge computing and federated learning.